

23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : C8

Team Members :

Roll No	Student Name
AM.SC.U4CSE25218	GORRELA TULASI LASYA
AM.SC.U4CSE25220	HANSIKA L CHAWLA
AM.SC.U4CSE25234	MUMMADI MANJUNADHA REDDY
AM.SC.U4CSE25236	NANDANA JAYAKUMAR

Project Title : CineHall - An Online Movie Ticket Booking System

3.1. Detailed description of the project

CineHall is an online movie ticket booking platform designed for smooth ticket reservations, showtime scheduling, and theater management. There are 3 main user roles: Customers, Theater Admins and System Administrators.

1.Core System Entities & Attributes

Entity	Description	Attributes
User	Represents any registered individual in the system.	UserID (Primary Key), Name, Email, Phone, PasswordHash, Role (Customer, Theater Admin, System Admin).
Movie	Represents films available for screening.	MovieID (Primary Key), Title, Genre, Duration (in minutes), Language, ReleaseDate, Rating.
Theater	Represents the physical cinema locations hosting screenings.	TheaterID (Primary Key), Name, City, Address, TotalScreens.
Screen	Represents individual auditoriums within a theater.	ScreenID (Primary Key), ScreenNumber, TotalSeats, ScreenType
Seat	Represents physical seating units inside a screen.	SeatID (Primary Key), SeatNumber, RowLetter, SeatType

Showtime	Represents a specific screening of a movie on a screen at a scheduled time.	ShowtimeID (Primary Key), ShowDate, StartTime, EndTime, BasePrice.
Booking	Represents a ticket reservation transaction initiated by a customer.	BookingID (Primary Key), BookingDate, TotalAmount, Status
Ticket	Represents individual seats reserved within a booking.	TicketID (Primary Key), SeatPrice, TicketStatus
Payment	Represents the transaction processing for a booking.	PaymentID (Primary Key), PaymentDate, Amount, PaymentMethod, PaymentStatus

2. Entity Relationships

- **User to Booking:** A **Customer** (User) can place **zero or many** Bookings, but each Booking belongs to **exactly one** Customer.
- **Movie to Showtime:** A **Movie** can have **zero or many** Showtimes scheduled, but each Showtime is associated with **exactly one** Movie.
- **Theater to Screen:** A **Theater** contains **one or many** Screens, and a Screen belongs to **exactly one** Theater.
- **Screen to Seat:** A **Screen** contains **one or many** Seats, and a Seat belongs to **exactly one** Screen.
- **Screen to Showtime:** A **Screen** hosts **zero or many** Showtimes, while a Showtime takes place on **exactly one** Screen.
- **Booking to Ticket:** A **Booking** can contain **one or many** Tickets, but each Ticket belongs to **exactly one** Booking.
- **Showtime & Seat to Ticket:** A **Ticket** links **exactly one** Showtime and **exactly one** Seat.

- **Booking to Payment:** A **Booking** generates **one** Payment transaction (or zero if pending), and a Payment belongs to **exactly one** Booking.

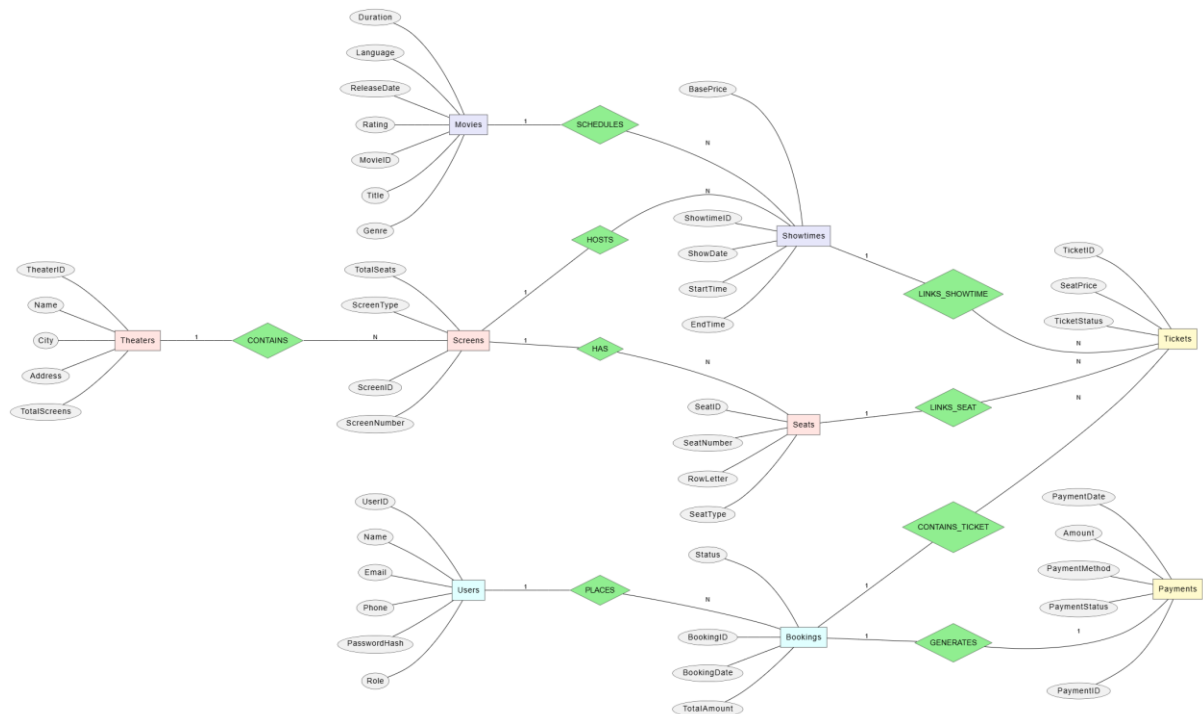
3. User Roles & System Interactions

1. **Customer:** Browses available movies, views theater schedules, selects available seats for a specific showtime, makes payments, receives confirmed tickets, and manages booking history.
2. **Theater Admin:** Manages screens, defines seating layouts, configures ticket prices, and schedules movie showtimes for their assigned theaters.
3. **System Admin:** Oversees platform operations, manages user accounts, updates movie listings, and monitors global system analytics.

4. Business Rules

1. **Unique Account Requirement:** Every user must register with a unique email address and phone number.
2. **No Overlapping Showtimes:** A screen cannot host two overlapping showtimes. The interval between StartTime and EndTime must accommodate the movie's duration plus a fixed cleaning buffer.
3. **Double-Booking Prevention:** A seat for a specific showtime can only be booked by **one** customer at a time. Once selected and paid for, its status changes to Occupied for that showtime.
4. **Pricing Logic:** Ticket prices are computed dynamically based on the BasePrice of the showtime and additional surcharges defined by the SeatType (VIP seats cost more than Standard seats).
5. **Payment Deadline:** Bookings remain in a Pending state for a maximum of 10 minutes. If payment is not completed within this window, reserved seats are automatically released back to the available pool.
6. **Cancellation & Refunds:** Bookings can only be cancelled up to 2 hours before the scheduled StartTime. Upon valid cancellation, the associated tickets become available for re-booking.

3.2. ER Diagram



3.3 Relational Schema

1. Customer Table

Column Name	Data Type	Constraints	Description
customer_id	SERIAL	PK	Unique identifier for the customer
name	VARCHAR(100)	NOT NULL	Customer's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Unique login email
phone	VARCHAR(15)	UNIQUE, NOT NULL	Contact phone number
password	VARCHAR(255)	NOT NULL	Secure password

2. Theatre Table

Column Name	Data Type	Constraints	Description
-------------	-----------	-------------	-------------

theatre_id	SERIAL	PK	Unique identifier for the theatre
name	VARCHAR(100)	NOT NULL	Theatre name
location	VARCHAR(255)	NOT NULL	Address location
contact_info	VARCHAR(50)	NOT NULL	Support contact info
total_screens	INT	NOT NULL	Number of screens

3. Screen Table

Column Name	Data Type	Constraints	Description
screen_id	SERIAL	PK	Unique identifier for the screen
theatre_id	INT	FK (Theatre)	Parent theatre reference
screen_number	INT	NOT NULL	Screen number
seating_capacity	INT	NOT NULL	Total seats available

4. Seat Table

Column Name	Data Type	Constraints	Description
seat_id	SERIAL	PK	Unique identifier for the seat
screen_id	INT	FK (Screen)	Parent screen reference
seat_row	VARCHAR(2)	NOT NULL	Row identifier (e.g., A, B)
seat_number	INT	NOT NULL	Seat number in row
seat_type	VARCHAR(20)	NOT NULL	Tier (Silver, Gold, Premium)

5. Movie Table

Column Name	Data Type	Constraints	Description
movie_id	SERIAL	PK	Unique identifier for the movie
title	VARCHAR(150)	NOT NULL	Movie title

genre	VARCHAR(50)	NOT NULL	Movie genre
language	VARCHAR(50)	NOT NULL	Language
duration	INT	NOT NULL	Duration in minutes
certificate	VARCHAR(10)	NOT NULL	Censor rating (U, UA, A)
status	VARCHAR(20)	NOT NULL	Status (Now Showing / Coming Soon)
synopsis	TEXT		Plot summary
rating	DECIMAL(3,1)		Average rating

6. Showtime Table

Column Name	Data Type	Constraints	Description
showtime_id	SERIAL	PK	Unique identifier for the showtime
movie_id	INT	FK (Movie)	Movie reference
screen_id	INT	FK (Screen)	Screen reference
show_date	DATE	NOT NULL	Screening date
show_time	TIME	NOT NULL	Screening time
silver_price	DECIMAL(8,2)	NOT NULL	Price for Silver seats
gold_price	DECIMAL(8,2)	NOT NULL	Price for Gold seats
premium_price	DECIMAL(8,2)	NOT NULL	Price for Premium seats

7. Booking Table

Column Name	Data Type	Constraints	Description
booking_id	SERIAL	PK	Unique identifier for the booking
customer_id	INT	FK (Customer)	Customer reference

showtime_id	INT	FK (Showtime)	Showtime reference
ticket_code	VARCHAR(20)	UNIQUE, NOT NULL	Generated unique ticket string
booking_time	TIMESTAMP	NOT NULL	Timestamp of booking
total_amount	DECIMAL(8,2)	NOT NULL	Total cost
status	VARCHAR(20)	NOT NULL	Status (Confirmed, Cancelled)

8. Booking_Seat Table

Column Name	Data Type	Constraints	Description
booking_seat_id	SERIAL	PK	Unique identifier
booking_id	INT	FK (Booking)	Booking reference
seat_id	INT	FK (Seat)	Seat reference

9. Payment Table

Column Name	Data Type	Constraints	Description
payment_id	SERIAL	PK	Unique identifier for payment
booking_id	INT	FK (Booking)	Booking reference
amount	DECIMAL(8,2)	NOT NULL	Transaction amount
payment_method	VARCHAR(30)	NOT NULL	Method (UPI, Card, etc.)
payment_status	VARCHAR(20)	NOT NULL	Status (Success, Failed)
payment_time	TIMESTAMP	NOT NULL	Timestamp

10. Cancellation Table

Column Name	Data Type	Constraints	Description
-------------	-----------	-------------	-------------

cancellation_id	SERIAL	PK	Unique identifier for cancellation
booking_id	INT	FK (Booking)	Booking reference
refund_status	VARCHAR(20)	NOT NULL	Refund status
reason	TEXT		Reason for cancellation
cancelled_at	TIMESTAMP	NOT NULL	Timestamp of cancellation

3.4 Normalization and Functional dependency analysis

To show normalisation we consider the UNF of this schema:

CineHall(booking_id, customer_id, theatre_id, screen_id, movie_id, showtime_id, seat_id, payment_id, cancellation_id, customer_name, email, phone, password, theatre_name, location, contact_info, total_screens, screen_number, seating_capacity, title, genre, language, duration, certificate, movie_status, synopsis, rating, show_date, show_time, silver_price, gold_price, premium_price, ticket_code, booking_time, total_amount, booking_status,

{seat_row, seat_number, seat_type},

{payment_amount, payment_method, payment_status, payment_time}, {refund_status, cancellation_reason, cancelled_at})

The UNF contains booking, customer, theatre, screen, movie, showtime, seat, payment and cancellation information in a single structure. This causes considerable redundancy because the same customer, theatre, movie or seat information can occur in multiple booking records.

FUNCTIONAL DEPENDENCIES:

	Function Dependency	Significance
FD1	Customer_id-> customer_name, email , phone, password	Ensures a customer's personal details are uniquely tied to their ID, centralizing profile management and preventing redundant data entry across multiple bookings.

FD2	Booking_seat_id->booking_id,seat_id	Resolves the many-to-many relationship between bookings and seats by mapping a specific reserved ticket strictly to its parent transaction and physical seat.
FD3	booking_id -> booking_time, total_amount, booking_status, customer_id, showtime_id, ticket_code	Centralizes all transactional details of a booking to its unique ID, ensuring transaction integrity and establishing the core link between the customer and the showtime.
FD4	showtime_id-> show_date, show_time, silver_price, gold_price, premium_price	Guarantees that a specific showtime dictates exactly when and where a movie plays, along with its dynamic pricing tiers, ensuring scheduling consistency.
FD5	movie_id-> title, genre, language, duration, certificate, movie_status, synopsis, rating	Isolates static movie data to a single identifier, meaning movie details are stored exactly once and referenced by ID, preventing update anomalies.
FD6	screen_id -> screen_number, seating_capacity	Links a specific physical screen to its capacity and parent theatre, ensuring accurate seating limits and location routing for the booking system.
FD7	Theatre_id -> theatre_name, location, contact_info, total_screens	Centralizes theatre information, preventing the duplication of addresses and contact info across multiple screens and showtimes.
FD8	Seat_id -> seat_row, seat_number, seat_type	Defines the physical properties and tier of a single seat within a specific screen, serving as the basis for double-booking prevention and dynamic pricing.
FD9	Payment_id -> payment_amount, payment_method, payment_status, booking_id	Ties payment gateway data to a specific transaction ID, ensuring accurate financial tracking independent of the booking itself.
FD10	Cancellation_id -> booking_id, refund_status, reason, cancelled_at	Isolates cancellation data to a specific event, keeping the

		main booking table clean while tracking refund states and chronological timestamps.
FD11	Email -> customer_id	Since every email is unique, it can serve as candidate key
FD12	ticket_code -> booking_id, customer_id, showtime_id, booking_time, total_amount, status	Guarantees that a generated ticket string can uniquely reverse-lookup the entire booking transaction, acting as a candidate key.

Candidate Key Justification Using Attribute Closure

For the booking-level UNF record, booking_id is the natural identifier of a booking.

The closure of {booking_id} is

$$\{\text{booking_id}\}^+ = \{\text{booking_id}\}$$

Using the functional dependency:

booking_id → customer_id, showtime_id, ticket_code, booking_time, total_amount, status

Therefore:

$$\{\text{booking_id}\}^+ = \{\text{booking_id}, \text{customer_id}, \text{showtime_id}, \text{ticket_code}, \text{booking_time}, \text{total_amount}, \text{status}\}$$

Since customer_id and showtime_id identify the associated customer and showtime, their dependencies, by transitivity of functional dependencies 1,5,3,7 the closure of booking_id is all attributes related to a booking.

$$\{\text{booking_id}\}^+ = \text{all attributes in UNF}$$

Hence, {booking_id} is a superkey for the booking-level UNF record. Since it consists of only one attribute, it is minimal and is therefore a candidate key.

The repeating seat group is deliberately not treated as a separate key in the UNF because the UNF is not yet a proper first-normal-form relation. The repeating group is separated during the 1NF decomposition into SEAT and BOOKING_SEAT.

The final schema also defines email and phone as unique in CUSTOMER, and ticket_code as unique in BOOKING, giving them candidate-key status within their respective relations.

First Normal Form (1NF)

A relation is in **1NF** when every attribute contains atomic values and repeating groups are removed. This UNF violates 1NF because it contains the repeating groups which are {seat_row, seat_number, seat_type}, {amount, payment_method, payment_status, payment_time}, {refund_status, reason, cancelled_at}. To make the relation 1NF we decompose the UNF.

Decomposing to remove repeating groups:

SEAT (seat_id, screen_id, seat_row, seat_number, seat_type)

BOOKING_SEAT (booking_seat_id, booking_id, seat_id)

PAYMENT (payment_id, booking_id, amount, payment_method, payment_status, payment_time)

CANCELLATION (cancellation_id, booking_id, refund_status, reason, cancelled_at)

The resulting 1NF relation is after decomposition is

the resulting 1NF relation after decomposition is CineHall (booking_id, customer_id, customer_name, email, phone, password, theatre_id, theatre_name, location, contact_info, total_screens, screen_id, screen_number, seating_capacity, movie_id, title, genre, language, duration, certificate, movie_status, synopsis, rating, showtime_id, show_date, show_time, silver_price, gold_price, premium_price, ticket_code, booking_time, total_amount, booking_status)

Second Normal Form (2NF)

A relation is in 2NF if it is already in 1NF and every non-key attribute is fully functionally dependent on the whole candidate key, no partial dependencies. The 1NF relation has single-attribute primary key like customer_id, theatre_id, screen_id, seat_id, movie_id, showtime_id, booking_id, booking_seat_id, payment_id, cancellation_id, which make having partial dependencies impossible.

The decomposed relations satisfies 2NF.

Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and no non-key attribute is transitively dependent on a candidate key. The 2NF relation contains several transitive dependencies.

This decomposition removes transitive dependencies such as:

booking_id -> customer_id -> customer details

booking_id -> showtime_id -> movie_id -> movie details

booking_id ->showtime_id ->screen_id ->theatre_id ->theatre details

booking_id ->showtime_id ->screen_id ->all seat attributes

The decomposition removes these transitive dependencies:

- CUSTOMER (customer_id, name, email, phone, password)
- THEATRE (theatre_id, name, location, contact_info, total_screens)
- SCREEN (screen_id, theatre_id, screen_number, seating_capacity)
- SEAT (seat_id, screen_id, seat_row, seat_number, seat_type)
- MOVIE (movie_id, title, genre, language, duration, certificate, status, synopsis, rating)
- SHOWTIME (showtime_id, movie_id, screen_id, show_date, show_time, silver_price, gold_price, premium_price)
- BOOKING (booking_id, customer_id, showtime_id, ticket_code, booking_time, total_amount, status)
- BOOKING_SEAT (booking_seat_id, booking_id, seat_id)
- PAYMENT (payment_id, booking_id, amount, payment_method, payment_status, payment_time)
- CANCELLATION (cancellation_id, booking_id, refund_status, reason, cancelled_at)

The relations satisfy 3NF.

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. The final CineHall relations satisfy this condition under the identified functional dependencies:

Relation	Main Functional Dependency	Determinant is a Superkey?
CUSTOMER	customer_id $\rightarrow$ name, email, phone, password	Yes
THEATRE	theatre_id $\rightarrow$ name, location, contact_info, total_screens	Yes
SCREEN	screen_id $\rightarrow$ theatre_id, screen_number, seating_capacity	Yes
SEAT	seat_id $\rightarrow$ screen_id, seat_row, seat_number, seat_type	Yes
MOVIE	movie_id $\rightarrow$ title, genre, language, duration, certificate, status, synopsis, rating	Yes
SHOWTIME	showtime_id $\rightarrow$ movie_id, screen_id, show_date, show_time, silver_price, gold_price, premium_price	Yes
BOOKING	booking_id $\rightarrow$ customer_id, showtime_id, ticket_code, booking_time, total_amount, status	Yes
BOOKING_SEAT	booking_seat_id $\rightarrow$ booking_id, seat_id	Yes
PAYMENT	payment_id $\rightarrow$ booking_id, amount, payment_method, payment_status, payment_time	Yes

CANCELLATION	cancellation_id → booking_id, refund_status, reason, cancelled_at	Yes
---------------------	--	-----

The relation satisfies BCNF.

Forth Normal Form

The relations satisfy 4NF because, after the decomposition up to BCNF, there are no non-trivial multivalued dependencies where the determinant is not a superkey. The independent repeating information for seats, payments, and cancellations has already been separated into their respective relations, so no further decomposition is required.

The relation satisfy 4NF.

Fifth Normal Form

A relation is in 5NF, if every non-trivial join dependency is implied by the candidate keys of the relation.

There are no non-trivial join dependencies that need further decomposition. The relationships between customers, bookings, showtimes, seats, payments, movies, screens, and theatres have already been represented through separate relations connected by primary keys and foreign keys.

The final decomposition produces the ten relations CUSTOMER, THEATRE, SCREEN, SEAT, MOVIE, SHOWTIME, BOOKING, BOOKING_SEAT, PAYMENT, and CANCELLATION.